Nombre	Manuel Fernando Guzmán Arias	ID	4948560
Carrera	Ingeniería de Software y Minería de Datos	Fecha	2026-10-19

Repositorio: https://github.com/FerGuzman65/Lab-7-Niveles-de-prueba

LABORATORIO # 7 – NIVELES DE PRUEBA

Introducción

En este laboratorio, los estudiantes aplicarán pruebas unitarias, de integración y de sistema sobre un programa de gestión de biblioteca desarrollado en Python.
A través de esta práctica, los estudiantes desarrollarán habilidades en:

- **Validar métodos individuales de clases (unidad).**
- **Verificar la correcta interacción entre múltiples clases (integración).**
- **Probar el sistema en su conjunto, evaluando caminos completos de ejecución (sistema).**
- **Detectar inconsistencias lógicas, entradas inválidas y condiciones excepcionales.**

Este laboratorio refuerza la importancia de aplicar distintos niveles de prueba para garantizar la calidad y confiabilidad de un sistema software.

Objetivo

El objetivo principal del laboratorio es que los alumnos comprendan cómo aplicar niveles de prueba (unitarias, integración y sistema) a un software realista, detectando errores y validando su correcto funcionamiento.

Descripción del programa

El programa implementa un sistema de gestión de biblioteca en consola. Permite:

- **Registrar libros con título, autor y estado (disponible o prestado).**
- **Registrar usuarios con su ID y nombre.**
- **Permitir préstamos de libros a usuarios, actualizando su estado.**
- **Permitir devoluciones de libros.**

- **Mostrar el listado de libros disponibles.**

Importante: El sistema inicialmente tiene todos los libros siguientes disponibles es decir no han sido prestados

Libros:

Título	ISBN
1984	ISBN001
1984	ISBN001
To Kill a Mockingbird	ISBN002
The Great Gatsby	ISBN003
Moby Dick	ISBN004
Pride and Prejudice	ISBN005
War and Peace	ISBN006

Requerimientos Funcionales y No Funcionales

Requerimientos Funcionales (RF)
• **RF1:** El sistema debe permitir registrar libros con título y autor.
• **RF2:** El sistema debe permitir registrar usuarios.
• **RF3:** El sistema debe permitir a un usuario pedir prestado un libro disponible.
• **RF4:** El sistema debe rechazar préstamos de libros ya prestados.
• **RF5:** El sistema debe permitir devolver libros.
• **RF6:** El sistema debe mostrar el listado de libros disponibles.
Requerimientos No Funcionales (RNF)
• **RNF1:** El sistema debe ser ejecutado en consola.
• **RNF2:** El sistema debe manejar errores de entrada de manera controlada.

Actividades

Instrucciones generales

Las siguientes actividades se llevarán a cabo **de manera sincronizada con el profesor**, quien explicará el procedimiento y las ventajas de cada paso:

1. Ingresa al enlace de **GitHub Classroom** proporcionado por el docente.
2. Abre el proyecto y revisa las funciones disponibles en los archivos proporcionados
3. Diseña los casos de prueba de acuerdo a las actividades.
4. Ejecuta las pruebas sobre el software asignado.

5. Documenta los resultados en las tablas que se muestran en la siguiente sección .
6. Entrega un breve análisis de los errores encontrados y una conclusión sobre la practica.
7. Realiza un commit por casos de pruebas es decir no es forzoso por cada caso para registrar tus resultados y observaciones en GitHub.

Actividad 1: Pruebas Unitarias:

- **Objetivo: Validar mínimo 4 métodos individuales de la clase que requiera.**

ID	**UT-01**
Entradas	Book('1984', 'George Orwell', 1949, 'ISBN001'); se llama book.borrow()
Descripción	Book.borrow(): un libro nuevo disponible pasa a prestado.
Resultado esperado	borrowed inicia en False y después de borrow() cambia a True.
Resultado obtenido	borrowed pasó de False a True. PASA.

ID	**UT-02**
Entradas	Mismo libro; se llama book.borrow() dos veces
Descripción	Book.borrow(): rechazo al prestar un libro que ya está prestado.
Resultado esperado	El segundo borrow() lanza ValueError('The book is already borrowed').
Resultado obtenido	Se lanzó ValueError con el mensaje esperado. PASA.

ID	**UT-03**
Entradas	Libro nuevo (no prestado); se llama book.return_book()

Descripción	Book.return_book(): no se puede devolver un libro que no está prestado.
Resultado esperado	Lanza ValueError('The book is not currently borrowed').
Resultado obtenido	Se lanzó ValueError con el mensaje esperado. PASA.

ID	**UT-04**
Entradas	User('Anna', 'U001'); register_borrow(libro '1984')
Descripción	User.register_borrow(): registra el préstamo en el historial.
Resultado esperado	get_history() regresa ['Borrowed: 1984'].
Resultado obtenido	Código original: regresaba ['Borrwed: 1984'] (error de ortografía en user.py). FALLA. Código corregido: PASA.

ID	**UT-05**
Entradas	User('Anna', 'U001'); register_return(libro '1984')
Descripción	User.register_return(): registra la devolución en el historial.
Resultado esperado	get_history() regresa ['Returned: 1984'].
Resultado obtenido	Código original: regresaba ['Reurned: 1984'] (error de ortografía en user.py). FALLA. Código corregido: PASA.

ID	**UT-06**
Entradas	Library con ISBN001; _find_book('ISBN999')
Descripción	Library._find_book(): búsqueda de un ISBN que no existe.
Resultado esperado	Lanza ValueError('Book not found').
Resultado obtenido	Se lanzó ValueError('Book not found'). PASA.

ID	UT-07
Entradas	Library con User('Anna', 'U001'); _find_user('U001')
Descripción	Library._find_user(): búsqueda de un usuario que sí existe.
Resultado esperado	Regresa el usuario y no imprime ningún mensaje de error.
Resultado obtenido	Código original: regresaba el usuario pero imprimía 'User not found'. FALLA. Código corregido: PASA.

ID	UT-08
Entradas	Library con '1984' (prestado) y 'Moby Dick'; get_available_books()
Descripción	Library.get_available_books(): listado de libros disponibles (RF6).
Resultado esperado	Regresa solo [Moby Dick].
Resultado obtenido	Código original: el método no existía (RF6 sin implementar). Código corregido: PASA.

Actividad 2: Pruebas de integración

Objetivo: Validar 2 interacción entre clases (User, Book, Library).

Ejemplos:

- Se agrega un libro y un usuario pide prestado el libro disponible.
- Se presta un libro y después se pide prestado para rechazo de préstamo de un libro ya prestado.
- Devolución correcta de libro y se solicita otro préstamo del mismo libro con otro usuario.

ID	IT-01

Entradas	Library + Book('1984', ISBN001) + User('Anna', U001); borrow_book('U001', 'ISBN001')
Descripción	Se agrega un libro y un usuario pide prestado el libro disponible.
Resultado esperado	El libro queda prestado y el historial de U001 es ['Borrowed: 1984'].
Resultado obtenido	Código original: el libro quedaba prestado, pero el historial decía ['Borrwed: 1984']. FALLA. Código corregido: PASA.

ID	**IT-02**
Entradas	U001 pide ISBN001; después U002 pide ISBN001
Descripción	Rechazo del préstamo de un libro que ya está prestado.
Resultado esperado	El segundo préstamo lanza ValueError('The book is already borrowed') y el historial de U002 queda vacío.
Resultado obtenido	Se rechazó el préstamo y el historial de U002 quedó vacío. PASA.

ID	**IT-03**
Entradas	U001 pide ISBN001, U001 lo devuelve, U002 pide ISBN001
Descripción	Devolución correcta y nuevo préstamo del mismo libro con otro usuario.
Resultado esperado	El libro queda prestado a U002; historial U001 = ['Borrowed: 1984', 'Returned: 1984'], U002 = ['Borrowed: 1984'].
Resultado obtenido	Código original: el flujo funcionaba, pero los historiales decían 'Borrwed' y 'Reurned'. FALLA. Código corregido: PASA.

ID	**IT-04**

Entradas	settings['loans_enabled'] = False; borrow_book('U001', 'ISBN001')
Descripción	Préstamos deshabilitados: el sistema debe rechazar el préstamo de forma controlada (RNF2).
Resultado esperado	Lanza ValueError('Loans are disabled'); el libro sigue disponible y el historial vacío.
Resultado obtenido	Código original: solo imprimía un mensaje y regresaba None, y el menú decía 'Book successfully borrowed'. FALLA. Código corregido: PASA.

Actividad 3: Pruebas de sistema

Objetivo: Validar al menos 2 caminos completos del sistema (interacción con el sistema completo).

ID	**ST-01**
Entradas	Menú: 1 → ISBN001 → U001, luego 3 → U001, luego 0
Descripción	Camino completo: pedir prestado desde el menú y consultar el historial.
Resultado esperado	Imprime 'Book successfully borrowed' y el historial muestra 'Borrowed: 1984'.
Resultado obtenido	Código original: la opción 1 ejecutaba return_book() e imprimía 'Error: The book is not currently borrowed'. FALLA. Código corregido: PASA.

ID	**ST-02**

Entradas	Menú: 1 → ISBN002 → U002, luego 2 → ISBN002 → U002, luego 0
Descripción	Camino completo: préstamo y devolución del mismo libro desde el menú.
Resultado esperado	Imprime 'Book successfully borrowed' y luego 'Book successfully returned', sin errores.
Resultado obtenido	Código original: las opciones 1 y 2 estaban invertidas en main.py. FALLA. Código corregido: PASA.

ID	**ST-03**
Entradas	Menú: 9; luego 1 → ISBN999 → U001; luego 3 → U999; luego 0
Descripción	Camino completo con entradas inválidas (RNF2: manejo controlado de errores).
Resultado esperado	Imprime 'Invalid option', 'Error: Book not found' y 'Error: User not found' sin que el programa truene.
Resultado obtenido	Los tres errores se manejan y el programa termina normal. PASA.

ID	**ST-04**
Entradas	Menú: 1 → ISBN003 → U003, luego 4, luego 0
Descripción	Camino completo: prestar un libro y consultar el listado de disponibles (RF6).
Resultado esperado	El listado muestra '1984' y ya no muestra ISBN003.
Resultado obtenido	Código original: no existía la opción 4 del menú (RF6 sin implementar). Código corregido: PASA.

Actividad 4: Documentación

Entrega de este reporte de laboratorio

Agregar sección de anexos de capturas de pantalla por cada caso de prueba

Análisis de los errores encontrados

Primero se diseñaron 13 casos de prueba con pytest (7 unitarios, 3 de integración y 3 de sistema) y se ejecutaron sobre el código original del repositorio: 6 pasaron y 7 fallaron (Anexo 17). Los fallos y la revisión del código mostraron cinco defectos:

1. Opciones del menú invertidas (main.py). La opción 1 decía "Borrow a book" pero llamaba a return_book(), y la opción 2 decía "Return a book" pero llamaba a borrow_book(). Lo detectaron ST-01 y ST-02.
2. Errores de ortografía en el historial (user.py). Se guardaba "Borrwed" y "Reurned". Lo detectaron UT-04 y UT-05, y se propagaba a IT-01 e IT-03.
3. Mensaje falso en _find_user (library.py). El print("User not found") estaba dentro del if que encuentra al usuario. Lo detectó UT-07.
4. RF6 no estaba implementado: no había forma de ver los libros disponibles.
5. Préstamos deshabilitados sin error (library.py). Con settings["loans_enabled"] = False, borrow_book() solo imprimía un mensaje y regresaba None, así que el menú decía "Book successfully borrowed" (RNF2).

Correcciones

En clase se corrigieron user.py (texto del historial) y library.py (se quitó el mensaje falso y se agregaron get_books() y get_users()); también se agregó __eq__ a Book y User y el archivo tests/test_library.py con pruebas propias. En ese archivo la prueba test_library_init_ID_not_registered usaba el ID U001, que sí está registrado, por lo que nunca lanzaba el error; se cambió a U999. Además se corrigió el menú de main.py, se agregó get_available_books() con la opción 4 del menú (RF6) y borrow_book() ahora lanza ValueError("Loans are disabled"). Para cubrir esto se agregaron UT-08, IT-04 y ST-04.

Las pruebas se marcaron por nivel con los marcadores unit, integration y system registrados en pytest.ini. Con el código corregido pasan las 21 pruebas del repositorio (Anexos 18 y 19).

Conclusiones

Con este laboratorio vi que cada nivel de prueba encuentra errores distintos. Las pruebas unitarias señalaron exactamente la línea con el problema, como los errores de ortografía del historial o el mensaje falso de _find_user. Las de integración mostraron cómo un error pequeño en User se nota en el flujo completo de préstamo y devolución de Library. Las de sistema encontraron el error más grave, el menú con las opciones invertidas, que ninguna prueba unitaria habría encontrado porque cada clase por separado funciona bien. También aprendí a probar un programa de consola con monkeypatch, simulando lo que escribe el usuario en input(), para revisar caminos completos sin capturar los datos a mano. Hacer los tres niveles me dio más confianza en el resultado que hacer solo uno, y volver a correr las mismas pruebas después de corregir el código confirmó que los cambios arreglaron los errores sin romper lo que ya funcionaba.

Anexos

Anexo 1. Caso UT-01: test_ut01_book_borrow_marks_as_borrowed (código de la prueba y resultado en pytest).

Lab 7 - UT-01

```
tests/test_unit.py
# UT-01: un libro nuevo inicia disponible y borrow() lo marca como prestado
def test_ut01_book_borrow_marks_as_borrowed():
    book = make_book()
    assert book.borrowed is False
    book.borrow()
    assert book.borrowed is True

> pytest -v tests/test_unit.py::test_ut01_book_borrow_marks_as_borrowed
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut01_book_borrow_marks_as_borrowed PASSED       [100%]

============================== 1 passed in 0.03s ===============================
```

Anexo 2. Caso UT-02: test_ut02_book_borrow_twice_raises (código de la prueba y resultado en pytest).

Lab 7 - UT-02

```
tests/test_unit.py
# UT-02: borrow() sobre un libro ya prestado lanza ValueError
def test_ut02_book_borrow_twice_raises():
    book = make_book()
    book.borrow()
    with pytest.raises(ValueError, match="already borrowed"):
        book.borrow()

> pytest -v tests/test_unit.py::test_ut02_book_borrow_twice_raises
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut02_book_borrow_twice_raises PASSED            [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 3. Caso UT-03: test_ut03_book_return_not_borrowed_raises (código de la prueba y resultado en pytest).

Lab 7 - UT-03

```
tests/test_unit.py
# UT-03: return_book() sobre un libro no prestado lanza ValueError
def test_ut03_book_return_not_borrowed_raises():
    book = make_book()
    with pytest.raises(ValueError, match="not currently borrowed"):
        book.return_book()

> pytest -v tests/test_unit.py::test_ut03_book_return_not_borrowed_raises
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut03_book_return_not_borrowed_raises PASSED     [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 4. Caso UT-04: test_ut04_user_register_borrow_history_text (código de la prueba y resultado en pytest).

Lab 7 - UT-04

```
tests/test_unit.py
# UT-04: register_borrow() guarda el préstamo en el historial con el texto correcto
def test_ut04_user_register_borrow_history_text():
    user = User("Anna", "U001")
    user.register_borrow(make_book())
    assert user.get_history() == ["Borrowed: 1984"]

> pytest -v tests/test_unit.py::test_ut04_user_register_borrow_history_text
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut04_user_register_borrow_history_text PASSED   [100%]

============================== 1 passed in 0.03s ===============================
```

Anexo 5. Caso UT-05: test_ut05_user_register_return_history_text (código de la prueba y resultado en pytest).

Lab 7 - UT-05

```
tests/test_unit.py
# UT-05: register_return() guarda la devolución en el historial con el texto correcto
def test_ut05_user_register_return_history_text():
    user = User("Anna", "U001")
    user.register_return(make_book())
    assert user.get_history() == ["Returned: 1984"]

> pytest -v tests/test_unit.py::test_ut05_user_register_return_history_text
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut05_user_register_return_history_text PASSED   [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 6. Caso UT-06: test_ut06_library_find_book_not_found_raises (código de la prueba y resultado en pytest).

Lab 7 - UT-06

```
tests/test_unit.py
# UT-06: _find_book() con un ISBN inexistente lanza ValueError
def test_ut06_library_find_book_not_found_raises():
    lib = Library()
    lib.add_book(make_book())
    with pytest.raises(ValueError, match="Book not found"):
        lib._find_book("ISBN999")

> pytest -v tests/test_unit.py::test_ut06_library_find_book_not_found_raises
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut06_library_find_book_not_found_raises PASSED  [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 7. Caso UT-07: test_ut07_library_find_existing_user_no_error_message (código de la prueba y resultado en pytest).

Lab 7 - UT-07

```
tests/test_unit.py
# UT-07: _find_user() con un usuario existente lo regresa sin imprimir "User not found"
def test_ut07_library_find_existing_user_no_error_message(capsys):
    lib = Library()
    user = User("Anna", "U001")
    lib.add_user(user)
    assert lib._find_user("U001") is user
    assert "User not found" not in capsys.readouterr().out

> pytest -v tests/test_unit.py::test_ut07_library_find_existing_user_no_error_message
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut07_library_find_existing_user_no_error_message PASSED [100%]

============================== 1 passed in 0.03s ===============================
```

Anexo 8. Caso UT-08: test_ut08_library_get_available_books (código de la prueba y resultado en pytest).

Lab 7 - UT-08

```
tests/test_unit.py
# UT-08: get_available_books() regresa solo los libros que no están prestados (RF6)
def test_ut08_library_get_available_books():
    lib = Library()
    b1 = make_book()
    b2 = Book("Moby Dick", "Herman Melville", 1851, "ISBN004")
    lib.add_book(b1)
    lib.add_book(b2)
    b1.borrow()
    assert lib.get_available_books() == [b2]

> pytest -v tests/test_unit.py::test_ut08_library_get_available_books
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_unit.py::test_ut08_library_get_available_books PASSED        [100%]

============================== 1 passed in 0.03s ===============================
```

Anexo 9. Caso IT-01: test_it01_borrow_available_book (código de la prueba y resultado en pytest).

Lab 7 - IT-01

```
tests/test_integration.py
# IT-01: se agrega un libro y un usuario pide prestado el libro disponible
def test_it01_borrow_available_book(lib):
    lib.borrow_book("U001", "ISBN001")
    assert lib._find_book("ISBN001").borrowed is True
    assert lib._find_user("U001").get_history() == ["Borrowed: 1984"]

> pytest -v tests/test_integration.py::test_it01_borrow_available_book
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_integration.py::test_it01_borrow_available_book PASSED       [100%]

============================== 1 passed in 0.03s ===============================
```

Anexo 10. Caso IT-02: test_it02_reject_borrow_of_borrowed_book (código de la prueba y resultado en pytest).

Lab 7 - IT-02

```
tests/test_integration.py
# IT-02: se presta un libro y otro usuario intenta pedirlo -> rechazo
def test_it02_reject_borrow_of_borrowed_book(lib):
    lib.borrow_book("U001", "ISBN001")
    with pytest.raises(ValueError, match="already borrowed"):
        lib.borrow_book("U002", "ISBN001")
    assert lib._find_user("U002").get_history() == []

> pytest -v tests/test_integration.py::test_it02_reject_borrow_of_borrowed_book
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_integration.py::test_it02_reject_borrow_of_borrowed_book PASSED [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 11. Caso IT-03: test_it03_return_then_borrow_by_other_user (código de la prueba y resultado en pytest).

Lab 7 - IT-03

```
tests/test_integration.py
# IT-03: devolución correcta y nuevo préstamo del mismo libro con otro usuario
def test_it03_return_then_borrow_by_other_user(lib):
    lib.borrow_book("U001", "ISBN001")
    lib.return_book("U001", "ISBN001")
    lib.borrow_book("U002", "ISBN001")
    assert lib._find_book("ISBN001").borrowed is True
    assert lib._find_user("U001").get_history() == ["Borrowed: 1984", "Returned: 1984"]
    assert lib._find_user("U002").get_history() == ["Borrowed: 1984"]

> pytest -v tests/test_integration.py::test_it03_return_then_borrow_by_other_user
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_integration.py::test_it03_return_then_borrow_by_other_user PASSED [100%]

============================== 1 passed in 0.03s ===============================
```

Anexo 12. Caso IT-04: test_it04_loans_disabled_rejects_borrow (código de la prueba y resultado en pytest).

Lab 7 - IT-04

```
tests/test_integration.py
# IT-04: con los préstamos deshabilitados se rechaza el préstamo y el libro no cambia (RNF2)
def test_it04_loans_disabled_rejects_borrow(lib, monkeypatch):
    monkeypatch.setitem(library.settings, "loans_enabled", False)
    with pytest.raises(ValueError, match="Loans are disabled"):
        lib.borrow_book("U001", "ISBN001")
    assert lib._find_book("ISBN001").borrowed is False
    assert lib._find_user("U001").get_history() == []

> pytest -v tests/test_integration.py::test_it04_loans_disabled_rejects_borrow
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_integration.py::test_it04_loans_disabled_rejects_borrow PASSED [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 13. Caso ST-01: test_st01_borrow_from_menu_and_view_history (código de la prueba y resultado en pytest).

Lab 7 - ST-01

```
tests/test_system.py
# ST-01: opción 1 (Borrow) con libro disponible -> préstamo exitoso y aparece en el historial
def test_st01_borrow_from_menu_and_view_history(monkeypatch, capsys):
    out = run_with_inputs(monkeypatch, capsys, ["1", "ISBN001", "U001", "3", "U001", "0"])
    assert "Book successfully borrowed" in out
    assert "Borrowed: 1984" in out

> pytest -v tests/test_system.py::test_st01_borrow_from_menu_and_view_history
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_system.py::test_st01_borrow_from_menu_and_view_history PASSED [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 14. Caso ST-02: test_st02_borrow_then_return_from_menu (código de la prueba y resultado en pytest).

Lab 7 - ST-02

```
tests/test_system.py
# ST-02: préstamo (opción 1) y devolución (opción 2) del mismo libro
def test_st02_borrow_then_return_from_menu(monkeypatch, capsys):
    out = run_with_inputs(monkeypatch, capsys, ["1", "ISBN002", "U002", "2", "ISBN002", "U002", "0"])
    assert "Book successfully borrowed" in out
    assert "Book successfully returned" in out
    assert "Error" not in out

> pytest -v tests/test_system.py::test_st02_borrow_then_return_from_menu
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_system.py::test_st02_borrow_then_return_from_menu PASSED      [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 15. Caso ST-03: test_st03_invalid_inputs_are_handled (código de la prueba y resultado en pytest).

Lab 7 - ST-03

```
tests/test_system.py
# ST-03: entradas inválidas (opción, ISBN y usuario) se manejan sin que el programa truene (RNF2)
def test_st03_invalid_inputs_are_handled(monkeypatch, capsys):
    out = run_with_inputs(monkeypatch, capsys, ["9", "1", "ISBN999", "U001", "3", "U999", "0"])
    assert "Invalid option" in out
    assert "Error: Book not found" in out
    assert "Error: User not found" in out

> pytest -v tests/test_system.py::test_st03_invalid_inputs_are_handled
============================= test session starts ==============================
collecting ... collected 1 item

tests/test_system.py::test_st03_invalid_inputs_are_handled PASSED        [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 16. Caso ST-04: test_st04_available_books_listing (código de la prueba y resultado en pytest).

Lab 7 - ST-04

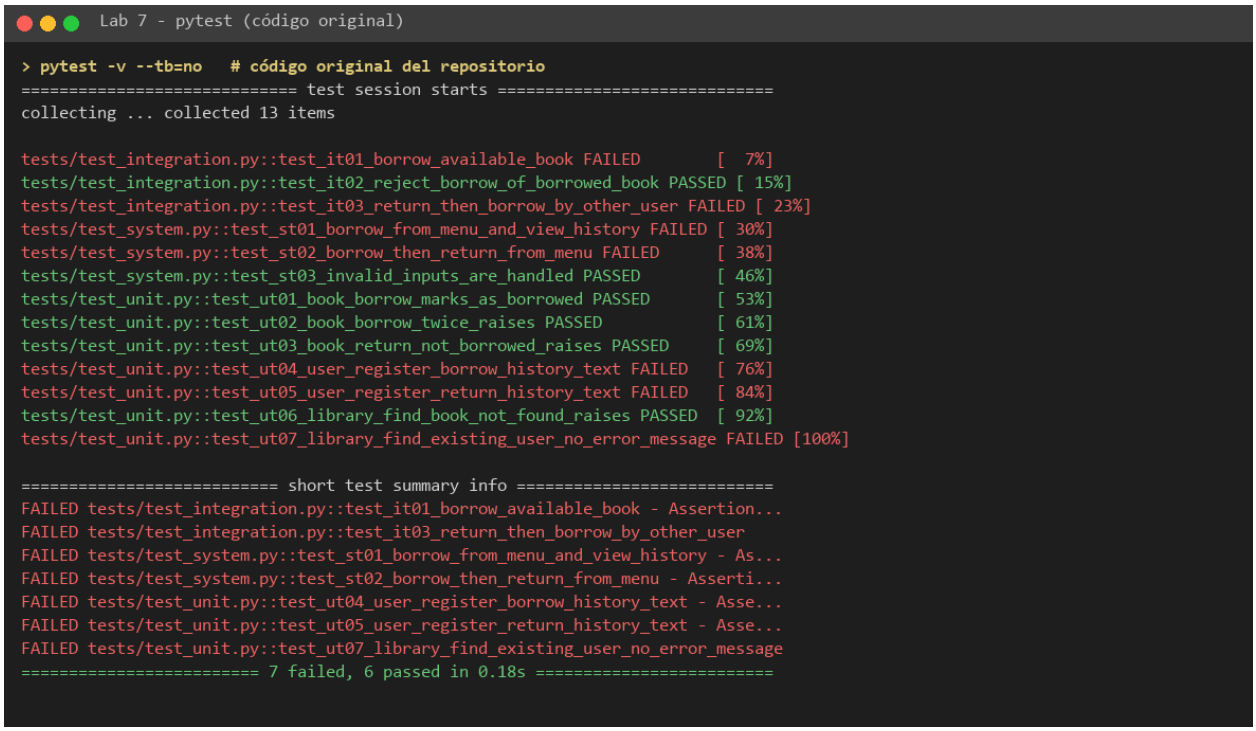
```
tests/test_system.py
# ST-04: después de prestar un libro, la opción 4 lo quita del listado de disponibles (RF6)
def test_st04_available_books_listing(monkeypatch, capsys):
    out = run_with_inputs(monkeypatch, capsys, ["1", "ISBN003", "U003", "4", "0"])
    listing = out.split("Available books:")[1]
    assert "ISBN001 - 1984" in listing
    assert "ISBN003" not in listing

> pytest -v tests/test_system.py::test_st04_available_books_listing
============================= test session starts ==============================
collecting ... collected 1 item

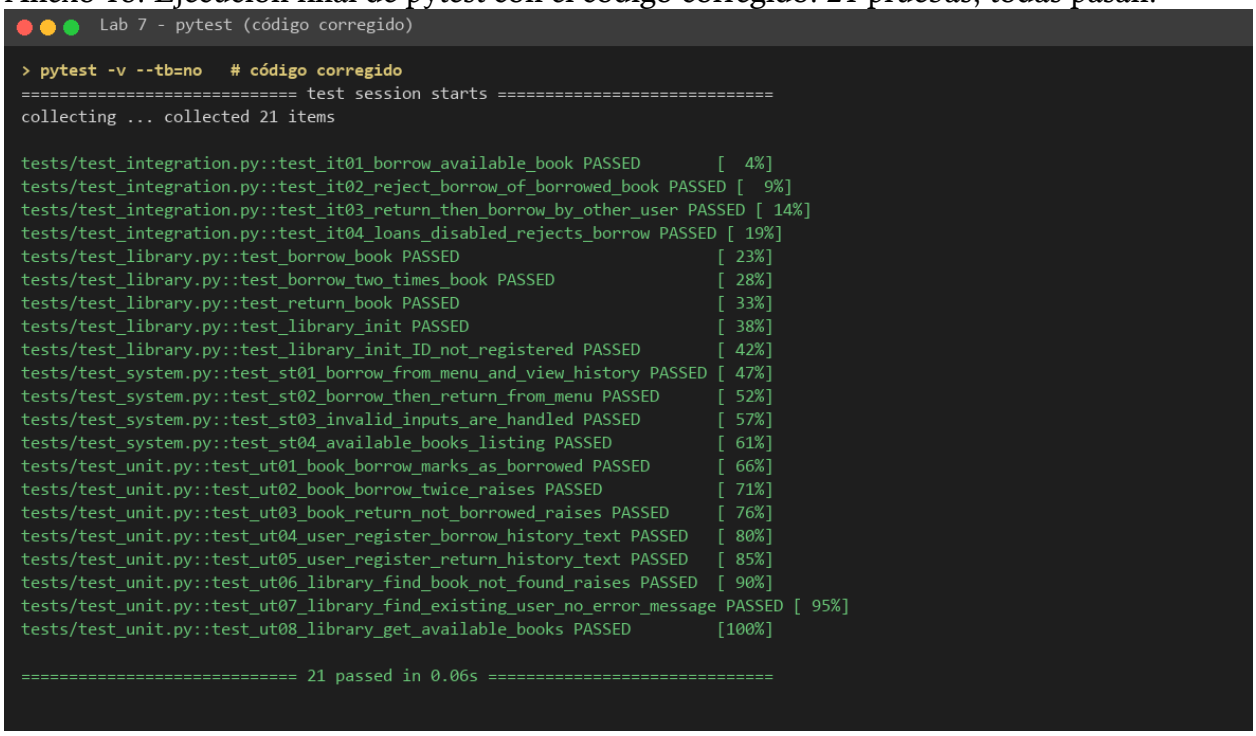
tests/test_system.py::test_st04_available_books_listing PASSED           [100%]

============================== 1 passed in 0.02s ===============================
```

Anexo 17. Primera ejecución de pytest sobre el código original: 13 casos, 6 pasan y 7 fallan.



Anexo 18. Ejecución final de pytest con el código corregido: 21 pruebas, todas pasan.



Anexo 19. Ejecución por nivel de prueba con los marcadores unit, integration y system.

